

Spark

The total time taken (main pipeline): 60.23s
Native speed step on full data: 0.41s
UDF on 1% sample: 0.99s | native on same sample: 0.08s
Driver RSS memory: 124.6 MB

spark

JobsStagesStorageEnvironmentExecutorsSQL / DataFrameSpark-NYC-Taxi application UI

Executors

>Show Additional Metrics

Summary

	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Excluded
Active(1)	0	1.3 MiB / 2.2 GiB	0.0 B	2	3	0	52	55	42 s (0.6 s)	604.8 MiB	1.1 GiB	1.9 GiB	0
Dead(0)	0	0.0 B / 0.0 B	0.0 B	0	0	0	0	0	0.0 ms (0.0 ms)	0.0 B	0.0 B	0.0 B	0
Total(1)	0	1.3 MiB / 2.2 GiB	0.0 B	2	3	0	52	55	42 s (0.6 s)	604.8 MiB	1.1 GiB	1.9 GiB	0

Executors

Show 20 entries

Search:

Executor ID	Address	Status	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Thread Dump	Heap Histogram	Ad...
driver	10.0.2.15:42421	Active	0	1.3 MiB / 2.2 GiB	0.0 B	2	3	0	52	55	42 s (0.6 s)	604.8 MiB	1.1 GiB	1.9 GiB	Thread Dump	Heap Histogram	200

Ray

The total time taken: 90.16s
UDF only time: 32.14s (35.7% of total)
Peak RSS memory: 2979.6 MB

Host / Worker Process name	State	State Message	ID	IP / PID	Actions	CPU	Memory
Ubuntu	ALIVE	-	72a65...	10.0.2.15 (Head)	Log	28.5%	7.74GB/10.86GB(71.3%)
python3	ALIVE	N/A	01000...	47458	Log CPU Flame Graph Stack Trace Memory Profiling	1.6%	323.25MB/10.86GB(2.9%)
ray::DashboardAgent	ALIVE	N/A		47887	Log CPU Flame Graph Stack Trace Memory Profiling	0.8%	209.61MB/10.86GB(1.9%)
ray::RuntimeEnvAgent	ALIVE	N/A		47889	Log CPU Flame Graph Stack Trace Memory Profiling	0%	128.82MB/10.86GB(1.2%)
ray::AutoscalingCoordinatorActor	ALIVE	N/A	f57a9...	47980	Log CPU Flame Graph Stack Trace Memory Profiling	0.6%	272.03MB/10.86GB(2.4%)

ray::MapBatches(cleanse_batch)- >MapBatches(add_features_and_speed)- >MapBatches(join_zones)->Write	ALIVE	N/A	f51e9...	48080	Log CPU Flame Graph Stack Trace Memory Profiling	99.8%	1.29GB/10.86GB(11.9%)
ray::MapBatches(cleanse_batch)- >MapBatches(add_features_and_speed)- >MapBatches(join_zones)->Write	ALIVE	N/A	9ba24...	48081	Log CPU Flame Graph Stack Trace Memory Profiling	80.7%	1.21GB/10.86GB(11.1%)
ray::BenchmarkTracker	ALIVE	N/A	675d0...	50194	Log CPU Flame Graph Stack Trace Memory Profiling	0.4%	152.10MB/10.86GB(1.4%)

- Ray's native Python execution model makes it more natural and productive for Python-centric ML workloads, but Spark's JVM foundation helps it have an edge over Ray for large-scale, structured data processing.
- One of the main issues of using Spark for python code is that it has to be translated into python code from the original JVM which takes longer and is inefficient.

Ray vs Spark

- In this case as we can see from the runtimes too, Spark is a better tool. This is because of its ability to process large amounts of data in a simpler way as compared to Ray.
- It is better for ingestion and heavy join functions because of its built in schema.

I used AI for the following updates

- A few specific configs in spark because my system did not have enough RAM in it to run the original case.
- I also used it to translate a bit of the pyspark code into ray.
- I also used it for the ctx part of the code to ensure it doesn't run into OOM errors.